

KVM-Based eBPF Rootkit Detection Engine & Toolkit

Requirements Document

Members:

Luis Abraham - labrahamesco2024@my.fit.edu

Dominick Morales - dmorales2024@my.fit.edu

Braiden Ames - bames2024@my.fit.edu

Dylin Irons - dirons2024@my.fit.edu

Advisor:

Dr. Ribeiro - eribeiro@fit.edu

Client:

Amy Alvarez - NASA Scientist

Florida Institute of Technology Senior Design

Table of Contents

- 1. Introduction**
 - 1.1. Purpose
 - 1.2. Scope
 - 1.3. Definitions, Acronyms, and Abbreviations
 - 1.4. References
- 2. Overall Description**
 - 2.1. Problem Description
 - 2.2. Operating Environment
 - 2.3. Design and Implementation Constraints
 - 2.4. Assumptions and Dependencies
- 3. External Interface Requirements**
 - 3.1. User Interfaces
 - 3.1.1. Command Line Interface
 - 3.1.2. Web Interface
 - 3.2. Software Interfaces
- 4. Functional Requirements**
 - 4.1. VM Management
 - 4.2. x86_64 VM Introspection
 - 4.3. ARM64 and pKVM Introspection
 - 4.4. VM Exit Monitoring
 - 4.5. Linux Kernel Introspection and Profiling
 - 4.6. eBPF Introspection and Telemetry
 - 4.7. VMI and Telemetry Correlation
 - 4.8. Malware Detection
 - 4.9. Research API
- 5. Other Non-Functional Requirements**
 - 5.1. Performance Requirements
 - 5.2. Software Quality Attributes
 - 5.3. Scalability and Maintainability
 - 5.4. Security and Isolation

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements of RootView, a virtual machine introspection platform designed to support the observation and eventual detection of malicious activity within the Linux kernel, with an emphasis on eBPF-based malware and rootkits.

The document establishes the expected capabilities, interfaces, constraints, and performance characteristics of the RootView system. It will serve as a reference for implementation, testing, and evaluation throughout the senior design project.

1.2 Scope

RootView is a research platform that uses virtualization-based introspection to observe Linux guest systems from outside the guest operating system. The platform aims to provide researchers with tools for examining guest memory, processor state, kernel structures, processes, and eBPF activity without relying exclusively on software executing within the monitored guest.

The system will support multiple virtualization architectures. On x86-64 systems, RootView will utilize KVM, QEMU, and KVMi to access and inspect guest state. On ARM64 systems, RootView will investigate and implement pKVM extensions that enable virtual machine introspection through the EL2 hypervisor environment.

RootView will also incorporate VM exit hooks to observe selected events occurring during guest execution. These observations will supplement traditional VMI capabilities and provide additional information about guest behavior.

The initial system will focus on establishing reliable VM management, introspection, and observation capabilities. Future development will expand these capabilities into a reusable research API and an eBPF malware detection engine that correlates hypervisor-level observations with guest kernel telemetry.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
VMI	Virtual Machine Introspection
KVM	Kernel-based Virtual Machine
KVMi	KVM Virtual Machine Introspection interface
pKVM	Protected KVM
EL2	ARM Exception Level 2, associated with hypervisor execution

eBPF	Extended Berkeley Packet Filter
BTF	BPF Type Format
DWARF	Debugging information format used to describe program and data structures
VM exit	A transition from guest execution to hypervisor execution
VM exit hook	A mechanism for observing or handling selected VM exit events
Guest	The virtual machine being monitored
Hypervisor	The virtualization layer responsible for managing guest execution
Kernel profile	Information used to interpret Linux kernel structures and symbols

1.4 References

- KVM documentation and implementation.
- LibVMI documentation.
- Linux KVM documentation.
- Linux pKVM documentation and relevant ARM virtualization documentation.
- Linux eBPF and BTF documentation.
- RootView project plan and design document.

2. Overall Description

2.1 Problem Description

Traditional malware detection tools frequently rely on software executing within the operating system being monitored. Malware operating with kernel-level privileges may interfere with these tools, conceal kernel objects, or manipulate the information reported to monitoring software.

RootView addresses this problem by providing an external observation point through the hypervisor. The system will inspect the guest's memory and execution state independently of the guest's own monitoring mechanisms.

The platform will initially target eBPF-based malware, which can use kernel execution capabilities to manipulate or conceal system activity. RootView will provide the infrastructure necessary to inspect eBPF-related kernel objects and correlate them with observations made at the virtualization layer.

The project will support both x86-64 and ARM64 environments, with architecture-specific introspection implementations.

2.2 Operating Environment

RootView is expected to operate in the following environments:

- Linux host systems supporting KVM virtualization.
- QEMU-managed virtual machines running supported Linux guest operating systems.
- x86-64 systems using KVM for guest introspection.
- ARM64 systems using pKVM and custom EL2 components for introspection.
- Unlocked ARM64 Android devices (e.g., Google Pixel 7) as the physical validation target for pKVM introspection, in addition to an emulated EL2 environment used for development.
- Linux guest kernels with available debugging or type information when required for kernel structure interpretation.

The host-side components will primarily be implemented in C/C++, with Python bindings and a web interface planned for higher-level interaction.

2.3 Design and Implementation Constraints

1. RootView must account for differences between x86-64 and ARM64 virtualization architectures.
2. KVM and pKVM provide different interfaces and execution environments, requiring architecture-specific implementations.
3. pKVM modifications must preserve the isolation and security properties expected of protected virtual machines.
4. Linux kernel structures may change between kernel versions, requiring adaptable kernel introspection mechanisms.
5. Some introspection capabilities may require kernel symbols, BTF/DWARF information, kernel profiles, or architecture-specific metadata.
6. Certain operations may require elevated host privileges or a modified kernel or hypervisor.
7. The introspection system must minimize interference with guest execution.
8. Because no existing introspection tooling equivalent to KVM exists for pKVM, RootView requires a custom kernel module to obtain EL2-level introspection access. This method is intended for research use only and is not safe or supported on production or locked-bootloader devices.

2.4 Assumptions and Dependencies

- The host system supports the required virtualization extensions.
- The user has sufficient privileges to configure and execute the virtualization environment.

- Supported guest kernels expose sufficient information for the required introspection operations.
- KVM and pKVM development environments are available for the respective architectures.
- The availability of kernel symbols and debugging information may vary between guest kernels.
- The detection engine will depend on the successful implementation of the underlying introspection and observation infrastructure.

3. External Interface Requirements

3.1 User Interfaces

RootView will provide interfaces for configuring and interacting with monitored virtual machines.

3.1.1 Command-Line Interface

The command-line interface shall allow users to:

- Create and configure virtual machines.
- Start, stop, and manage monitored virtual machines.
- Connect to a running guest for introspection.
- Request memory, register, process, and eBPF information.
- Configure and retrieve supported observation events.
- View errors and introspection results.

3.1.2 Web Interface

The web interface will provide a higher-level interface for interacting with RootView. It is expected to support VM management, display introspection results, and provide access to selected monitoring capabilities.

The web interface will depend on the functionality exposed by the backend.

3.2 Software Interfaces

RootView will interface with the following software components:

Component	Purpose
QEMU/KVM	VM creation, execution, and management
KVMI	x86-64 guest introspection
LibVMI	Existing VMI functionality and Linux introspection support

pKVM/EL2 modules	ARM64 introspection support
Linux kernel interfaces	Guest and host kernel functionality
BTF/DWARF	Kernel structure and type information
eBPF	Kernel telemetry and activity observation
C/C++ backend	Core introspection and VM management
Python	Higher-level research API and future detection engine
Web framework	User-facing interface

The system should provide an abstraction layer that allows higher-level components to interact with supported architectures without needing to directly manage architecture-specific introspection mechanisms.

4. Functional Requirements

4.1 VM Management

VM-001 through VM-005

RootView shall provide functionality for creating and managing virtual machines used for introspection.

- VM-001: The system shall allow users to configure supported virtual machines using a configuration file or command-line interface.
- VM-002: The system shall allow users to start and stop supported virtual machines.
- VM-003: The system shall allow the introspection backend to connect to a running virtual machine.
- VM-004: The system shall maintain the guest's execution state during supported introspection operations unless a pause or state transition is explicitly required.
- VM-005: The system shall report errors when a VM cannot be created, started, or accessed.

4.2 x86-64 Virtual Machine Introspection

VMI-001 through VMI-007

RootView shall support external introspection of x86-64 Linux guests using KVM and KVMi.

- VMI-001: The system shall retrieve physical memory from a monitored guest.

- VMI-002: The system shall support virtual-to-physical address translation.
- VMI-003: The system shall retrieve supported guest processor register state.
- VMI-004: The system shall provide access to guest memory without requiring an introspection agent inside the guest.
- VMI-005: The system shall support enumeration of basic Linux processes.
- VMI-006: The system shall provide access to selected Linux kernel structures.
- VMI-007: The system shall report failures encountered during memory access or address translation.

4.3 ARM64 and pKVM Introspection

ARM-001 through ARM-007

RootView shall investigate and implement introspection support for ARM64 virtual machines using pKVM and hypervisor-level components executing at EL2.

- ARM-001: The system shall provide an architecture-specific mechanism for accessing supported guest state through pKVM.
- ARM-002: The system shall support retrieval of guest memory for supported introspection operations.
- ARM-003: The system shall support retrieval of selected guest processor state.
- ARM-004: The system shall provide a mechanism for interpreting guest memory and address translation information on ARM64.
- ARM-005: The system shall provide an interface for communicating introspection information from EL2 components to the host-side backend.
- ARM-006: The system shall preserve the intended isolation properties of the pKVM environment.
- ARM-007: The system shall report unsupported operations and failures associated with ARM64 introspection.
- ARM-008: The system shall support deployment of the pKVM introspection module on physical, unlocked ARM64 Android hardware (e.g., Google Pixel 7) for validation beyond emulation.
- ARM-009: The system shall support development and initial validation of pKVM introspection components using an emulated EL2 environment prior to hardware deployment.
- ARM-010: The system shall document that its pKVM introspection method circumvents pKVM's intended isolation guarantees and is unsuitable for non-research or production devices.

4.4 VM Exit Monitoring

EXIT-001 through EXIT-006

RootView shall provide a mechanism for observing selected VM exit events through hypervisor-level hooks.

- EXIT-001: The system shall provide hooks for observing supported VM exit events.
- EXIT-002: The system shall retrieve relevant information associated with supported VM exit events.
- EXIT-003: The system shall allow supported exit events to be forwarded to the host-side introspection backend.
- EXIT-004: The system shall allow the backend to process exit-event information independently of the guest operating system.
- EXIT-005: The system shall provide a mechanism for enabling, disabling, or configuring supported exit-event monitoring where supported.
- EXIT-006: The system shall minimize unnecessary disruption to guest execution when monitoring VM exit events.

4.5 Linux Kernel Introspection and Profiling

LINUX-001 through LINUX-007

- LINUX-001: The system shall support retrieval of selected Linux kernel structures from a monitored guest.
- LINUX-002: The system shall support enumeration of guest processes using externally obtained kernel information.
- LINUX-003: The system shall investigate and utilize BTF/DWARF information to interpret kernel structures.
- LINUX-004: The system shall support kernel symbol resolution for supported guest kernels.
- LINUX-005: The system shall support address translation and page-table inspection for supported architectures.
- LINUX-006: The system shall identify unsupported or unavailable kernel metadata.
- LINUX-007: The system shall support testing against multiple Linux kernel versions.

4.6 eBPF Introspection and Telemetry

BPF-001 through BPF-007

- BPF-001: The system shall provide a mechanism for identifying eBPF programs within a monitored Linux guest.
- BPF-002: The system shall support inspection of selected eBPF program metadata.
- BPF-003: The system shall support inspection of selected eBPF maps and their associated metadata.
- BPF-004: The system shall support identification of selected eBPF program attachments.

- BPF-005: The system shall provide a mechanism for collecting supported eBPF-related events.
- BPF-006: The system shall represent collected events in a standardized format.
- BPF-007: The system shall provide access to eBPF introspection results through the backend interface.

4.7 VMI and Telemetry Correlation

CORR-001 through CORR-004

- CORR-001: The system shall provide a mechanism for correlating eBPF telemetry with VMI observations.
- CORR-002: The system shall allow the backend to compare observations from different monitoring sources.
- CORR-003: The system shall preserve relevant metadata, such as event type and timestamp, for supported observations.
- CORR-004: The system shall report discrepancies or inconsistencies identified by configured correlation logic.

4.8 Malware Detection

DETECT-001 through DETECT-005

- DETECT-001: The system shall provide an extensible interface for implementing eBPF malware detection techniques.
- DETECT-002: The system shall allow detection logic to consume VMI observations and eBPF telemetry.
- DETECT-003: The system shall support reporting suspicious activity identified by implemented detection logic.
- DETECT-004: The system shall provide information associated with detection results to assist researchers in analyzing suspicious behavior.
- DETECT-005: The system shall allow additional detection techniques to be incorporated without requiring major modifications to the underlying introspection backend.

4.9 Research API

API-001 through API-006

- API-001: The system shall provide a C/C++ interface for supported VM management and introspection operations.
- API-002: The system shall provide a Python interface to selected backend functionality.
- API-003: The API shall expose supported memory, Linux, and eBPF introspection operations.
- API-004: The API shall provide structured results for supported introspection operations.

- API-005: The API shall report errors through a consistent interface.
- API-006: The API shall provide an interface suitable for integration with future detection techniques.

5. Other Non-Functional Requirements

5.1 Performance Requirements

- PERF-001: Introspection operations shall minimize disruption to guest execution.
- PERF-002: VM exit monitoring shall minimize additional processing overhead.
- PERF-003: The system shall measure and document the performance impact of supported introspection operations.
- PERF-004: The system shall support repeated introspection operations during guest execution.
- PERF-005: The system shall document the performance differences between supported x86-64 and ARM64 implementations.

5.2 Software Quality Attributes

- QUAL-001: The system shall provide error handling for unsupported operations and failed introspection requests.
- QUAL-002: The system shall separate architecture-specific implementations from higher-level introspection interfaces where practical.
- QUAL-003: The system shall provide testing for supported memory, kernel, and eBPF introspection operations.
- QUAL-004: The system shall document supported host and guest configurations.
- QUAL-005: The system shall provide sufficient logging to assist with debugging and research.

5.3 Scalability and Maintainability

- MAINT-001: The backend shall support extending introspection capabilities without requiring a complete redesign.
- MAINT-002: The system shall support adding new architecture-specific introspection implementations.
- MAINT-003: The system shall support additional Linux kernel profiles and metadata sources.
- MAINT-004: The system shall support integrating additional eBPF detection techniques.
- MAINT-005: The system shall document the interfaces between the virtualization, introspection, telemetry, and detection components.

5.4 Security and Isolation

- SEC-001: The system shall limit access to introspection operations to authorized host-side components.
- SEC-002: Introspection components shall not require a privileged monitoring agent inside the guest for supported external VMI operations.
- SEC-003: ARM64 introspection extensions shall preserve the isolation guarantees required by the pKVM architecture.
- SEC-004: The system shall document any modifications to the hypervisor or kernel required for introspection.
- SEC-005: The system shall identify and document security assumptions associated with its introspection mechanisms.